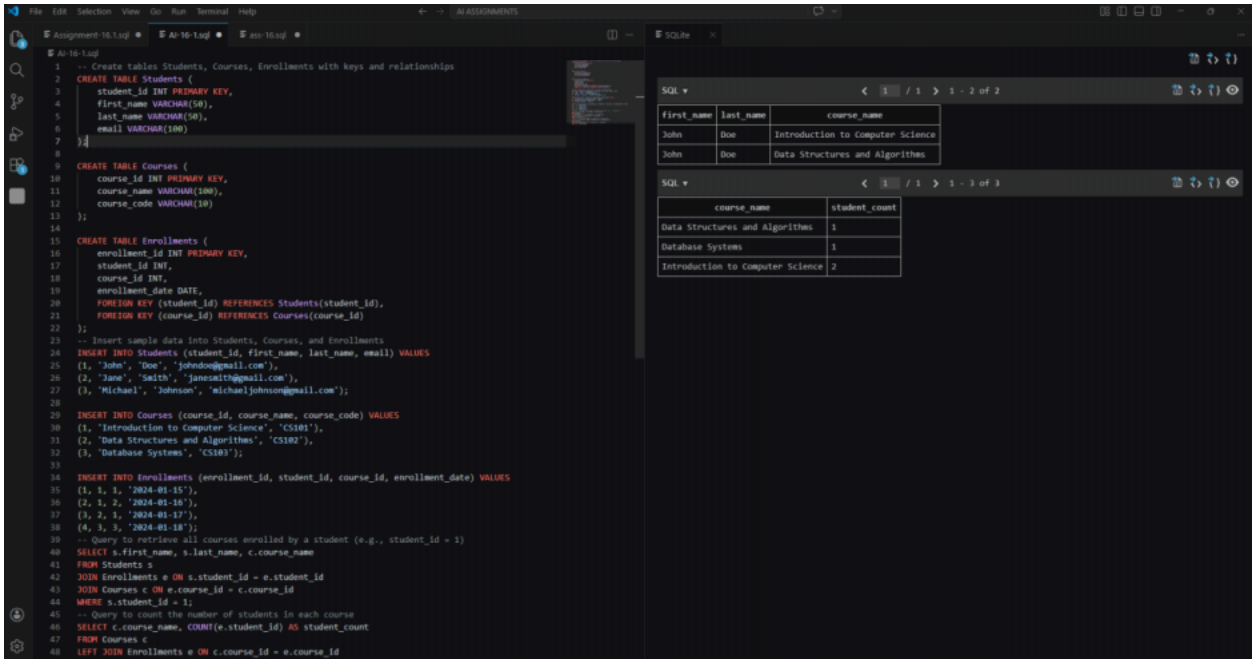
ASSIGNMENT-16.1

NAME: N.Blessy
ROLL NO: 2303A51062
BATCH: 29

TASK-1:

PROMPT:
--create tables Students, Courses, Enrollments, primary keys, foreign keys, and Relationships

CODE:



```sql
-- Create tables Students, Courses, Enrollments with keys and relationships
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    course_code VARCHAR(10)
);

CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
-- Insert sample data into Students, Courses, and Enrollments
INSERT INTO Students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'johndoe@gmail.com'),
(2, 'Jane', 'Smith', 'janesmith@gmail.com'),
(3, 'Michael', 'Johnson', 'michaeljohnson@gmail.com');

INSERT INTO Courses (course_id, course_name, course_code) VALUES
(1, 'Introduction to Computer Science', 'CS101'),
(2, 'Data Structures and Algorithms', 'CS102'),
(3, 'Database Systems', 'CS103');

INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date) VALUES
(1, 1, 1, '2024-01-15'),
(2, 1, 2, '2024-01-16'),
(3, 2, 1, '2024-01-17'),
(4, 3, 3, '2024-01-18');
-- Query to retrieve all courses enrolled by a student (e.g., student_id = 1)
SELECT s.first_name, s.last_name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE s.student_id = 1;
-- Query to count the number of students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
```

first_name	last_name	course_name
John	Doe	Introduction to Computer Science
John	Doe	Data Structures and Algorithms

course_name	student_count
Data Structures and Algorithms	1
Database Systems	1
Introduction to Computer Science	2

OBSERVATION:

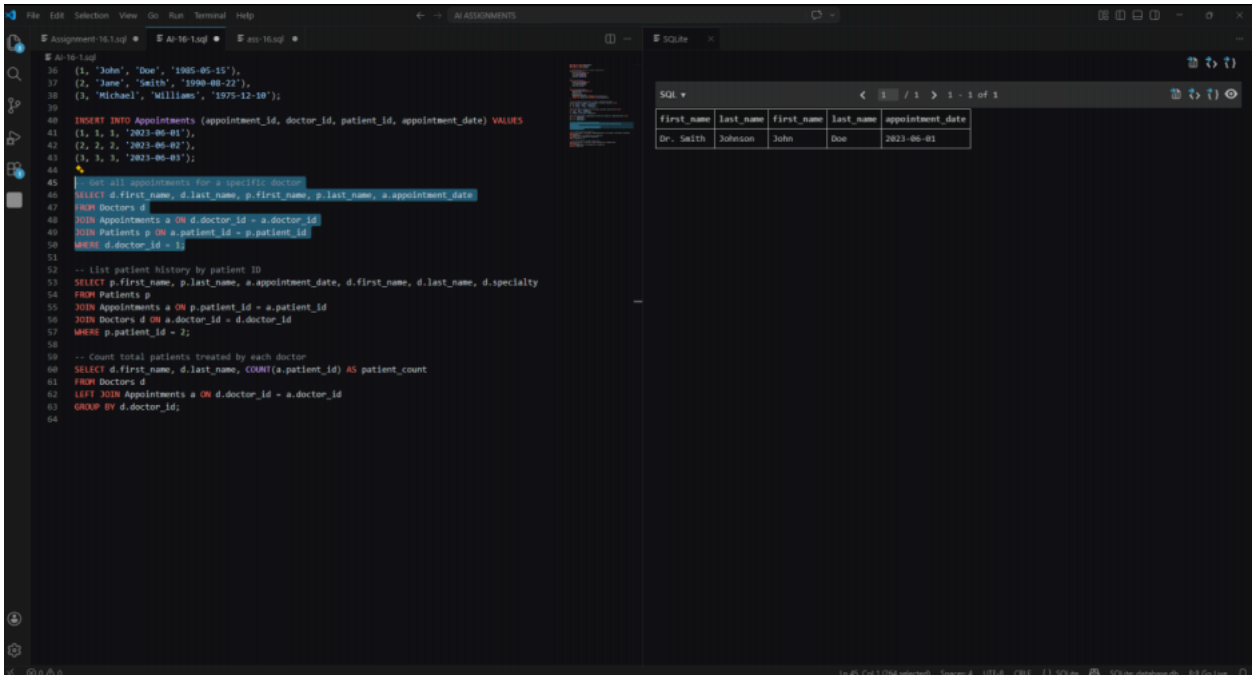
This task helped me understand how student and course data are stored in a database. I learned how to connect tables using primary key and foreign key. It also improved my knowledge of writing SQL queries to get useful information.

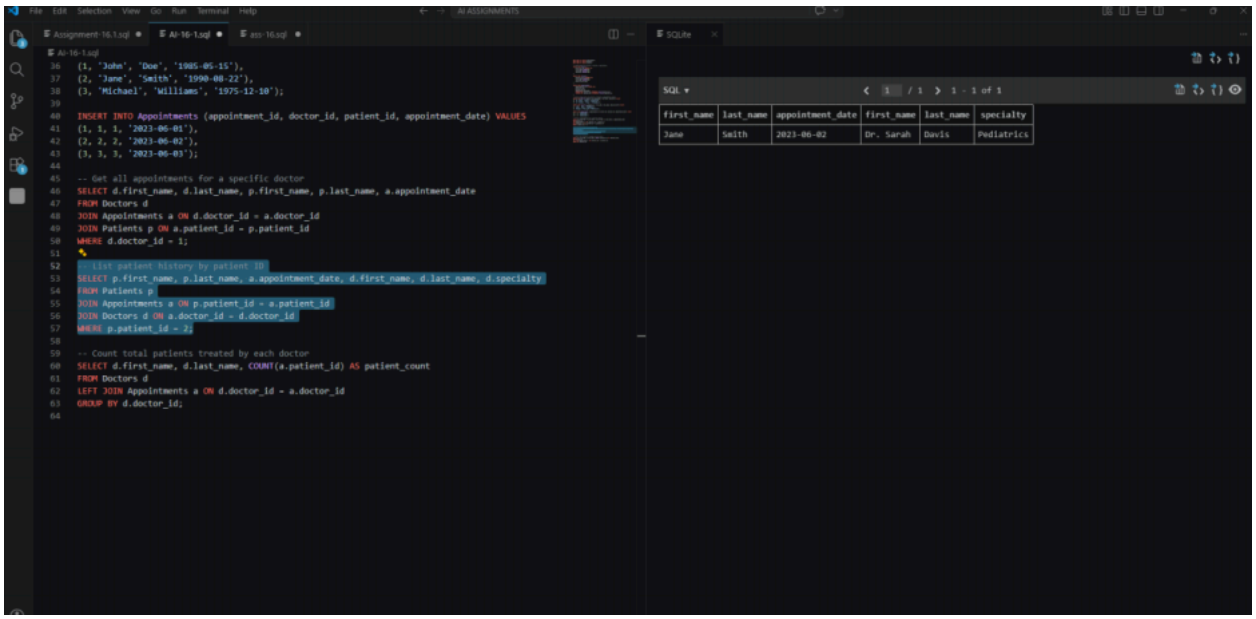
TASK-2:

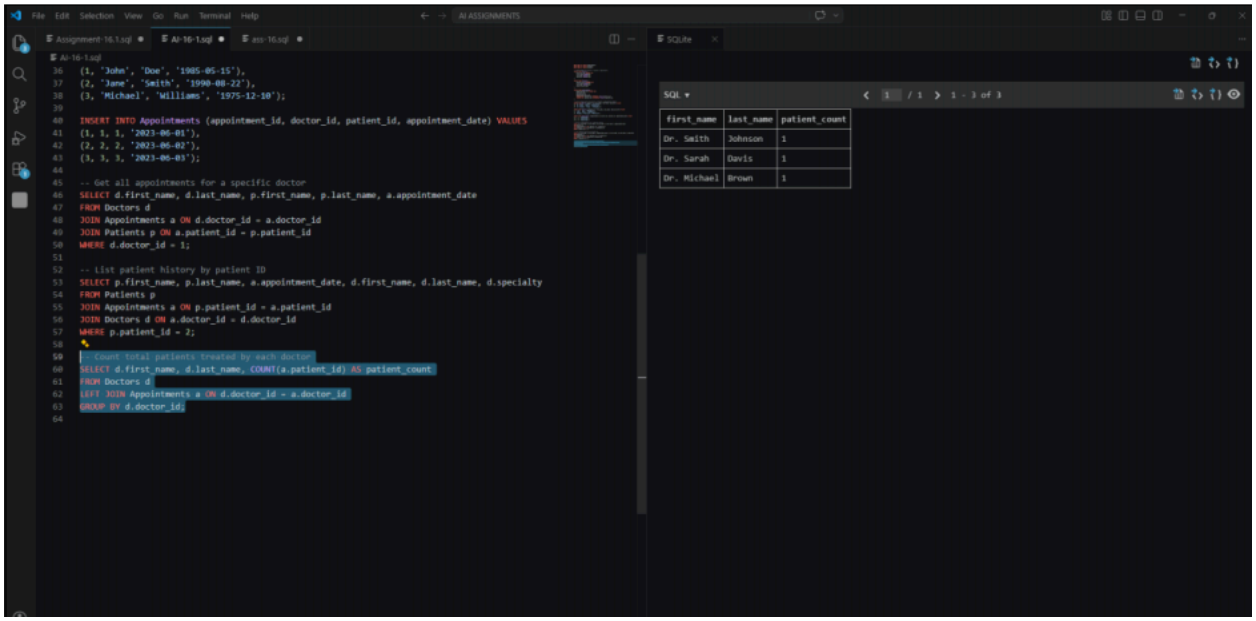
PROMPT:

--create tables doctors, patients, appointments, define consstraints(unique IDs,valid dates)

CODE:







OBSERVATION:

In this task, I understood how databases are used in hospitals to manage doctors, patients, and appointments. I learned about using constraints to avoid wrong data. It also helped me practice queries for real-life situations.

TASK-3:

PROMPT:
--create tables books,members,loans

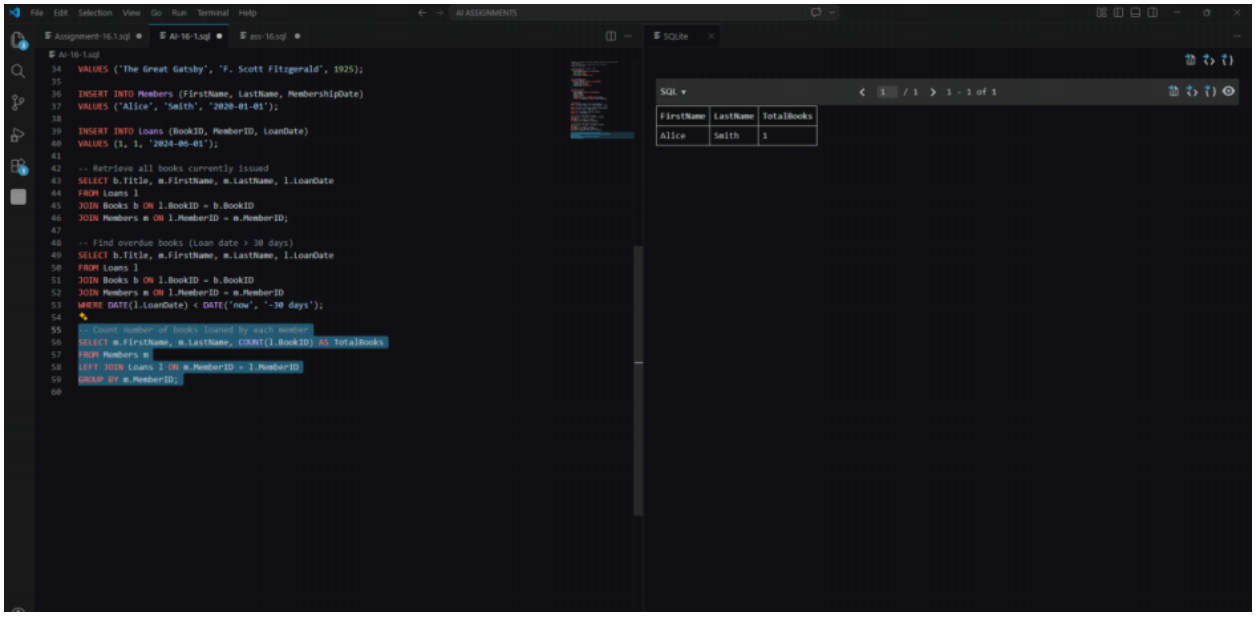
CODE:

```sql
CREATE TABLE Loans (
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- Insert data
INSERT INTO Books (Title, Author, PublishedYear)
VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);

INSERT INTO Members (FirstName, LastName, MembershipDate)
VALUES ('Alice', 'Smith', '2020-01-01');

INSERT INTO Loans (BookID, MemberID, LoanDate)
VALUES (1, 1, '2024-06-01');

-- Retrieve all books currently issued
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID;

-- Find overdue books (Loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE DATE(l.LoanDate) < DATE('now', '-30 days');

-- Count number of books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01


```sql
VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);

INSERT INTO Members (FirstName, LastName, MembershipDate)
VALUES ('Alice', 'Smith', '2020-01-01');

INSERT INTO Loans (BookID, MemberID, LoanDate)
VALUES (1, 1, '2024-06-01');

-- Retrieve all books currently issued
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID;

-- Find overdue books (Loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE DATE(l.LoanDate) < DATE('now', '-30 days');

-- Count number of books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Title	FirstName	LastName	LoanDate
The Great Gatsby	Alice	Smith	2024-06-01



OBSERVATION:

This task helped me learn how a library system works using a database. I understood how to track books and members. I also learned how indexing improves performance and how to find overdue books using queries.

TASK-4:

PROMPT:
--create tables Users, Products, Orders, OrderDetails.

CODE:

```sql
('Jane', 'Smith', 'janesmith@gmail.com');

INSERT INTO Products (ProductName, Price) VALUES
('Laptop', 999.99),
('Smartphone', 499.99);

INSERT INTO Orders (UserID, OrderDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-02');

INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 2, 1);

-- retrieve all orders by a user
SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
FROM Users u
JOIN Orders o ON u.UserID = o.UserID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE u.UserID = 1;

-- Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;

-- calculate total revenue in a given month
SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

SQL 1 / 1 1 - 2 of 2

FirstName	LastName	OrderDate	ProductName	Quantity
John	Doe	2024-06-01	Laptop	1
John	Doe	2024-06-01	Smartphone	2

```sql
INSERT INTO Orders (UserID, OrderDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-02');

INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 2, 1);

-- retrieve all orders by a user
SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
FROM Users u
JOIN Orders o ON u.UserID = o.UserID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE u.UserID = 1;

-- Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;

-- calculate total revenue in a given month
SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

SQL 1 / 1 1 - 1 of 1

ProductName	TotalQuantity
Smartphone	3

```sql
INSERT INTO Orders (UserID, OrderDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-02');

INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 2, 1);

-- retrieve all orders by a user
SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
FROM Users u
JOIN Orders o ON u.UserID = o.UserID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE u.UserID = 1;

-- Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 1;

-- calculate total revenue in a given month
SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

OrderMonth	TotalRevenue
2024-06	2499.96

OBSERVATION:
This task gave me an idea about how e-commerce websites store data. I learned how different tables like users, products, and orders are connected. Writing queries like total revenue helped me understand business analysis.

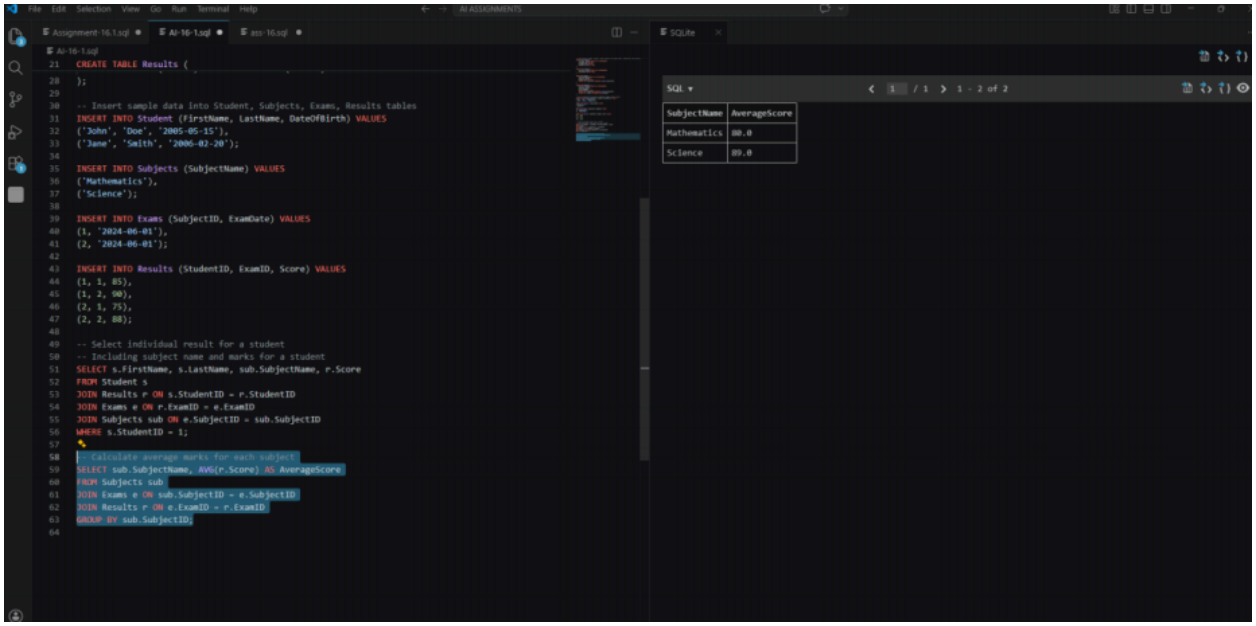
TASK-5:

PROMPT:
--create tables Student,Subjects,Exams,Results.primary keys, foreign keys, referential integrity constraints

CODE:

```sql
CREATE TABLE Results (
);

-- Insert sample data into Student, Subjects, Exams, Results tables
INSERT INTO Student (FirstName, LastName, DateOfBirth) VALUES
('John', 'Doe', '2005-05-15'),
('Jane', 'Smith', '2006-02-20');

INSERT INTO Subjects (SubjectName) VALUES
('Mathematics'),
('Science');

INSERT INTO Exams (SubjectID, ExamDate) VALUES
(1, '2024-06-01'),
(2, '2024-06-01');

INSERT INTO Results (StudentID, ExamID, Score) VALUES
(1, 1, 85),
(1, 2, 90),
(2, 1, 75),
(2, 2, 88);

-- Select individual result for a student
-- Including subject name and marks for a student
SELECT s.FirstName, s.LastName, sub.SubjectName, r.Score
FROM Student s
JOIN Results r ON s.StudentID = r.StudentID
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects sub ON e.SubjectID = sub.SubjectID
WHERE s.StudentID = 1;

-- Calculate average marks for each subject
SELECT sub.SubjectName, AVG(r.Score) AS AverageScore
FROM Subjects sub
JOIN Exams e ON sub.SubjectID = e.SubjectID
JOIN Results r ON e.ExamID = r.ExamID
GROUP BY sub.SubjectID;
```

FirstName	LastName	SubjectName	Score
John	Doe	Mathematics	85
John	Doe	Science	90



OBSERVATION:
This task helped me understand how exam data is managed. I learned how to store marks and calculate averages using SQL. It also improved my skills in using joins and aggregation functions.